CSL3050:Database Systems
Lab_Assignment-1

Soumen Kumar
B22ES006

Q9-Consider an institute that needs to manage its mess facilities efficiently for students across various academic levels (UG, PG, PhD) from different departments (Civil, Mechanical, Electrical, Computer Science, etc.). The institute has two types of messes: Veg and Non-Veg, each with three floors. Design an ER diagram that includes the necessary entities, relationships, and attributes to capture all relevant data. The system should capture details about the students, including their level and department. It should also record information about the messes, such as the mess type and floor number. The system must monitor the distribution of meals—categorized as breakfast, snacks, lunch, and dinner—served to students, ensuring that each transaction is logged with details like the date, time, and quantity of the meal served. Feel free to add any additional attributes or constraints that are essential for accurately representing the mess management system.

Answer-

Constraints

1. **Primary Key Constraint**:
   - Each entity must have a unique primary key (e.g., `Student_ID`, `Mess_ID`, `Meal_ID`, `Transaction_ID`) to ensure entity uniqueness.
2. **Foreign Key Constraints**:
   - `Student_ID` in the **Transaction** entity must exist in the **Student** entity.
   - `Mess_ID` in the **Transaction** entity must exist in the **Mess** entity.
   - `Meal_ID` in the **Transaction** entity must exist in the **Meal** entity.
3. **Unique Constraints**:
   - **Student** entity should ensure that each `Student_ID` is unique.
   - **Mess** entity should ensure that each `Mess_ID` is unique.
4. **Check Constraints**:
   - **Levels** in the **Student** entity should only accept values: UG, PG, PhD.
   - **Mess_Type** in the **Mess** entity should only accept values: Veg, Non-Veg.
   - **Floor_Number** in the **Mess** entity should only accept values: 1, 2, 3.
   - **Meal_Type** in the **Meal** entity should only accept values: Breakfast, Snacks, Lunch, Dinner.
5. **Not Null Constraints**:
   - Attributes like `Name`, `Level`, `Department`, `Date`, `Time`, and `Quantity` in the **Student**, **Transaction**, and **Meal** entities should not be null.
6. **Date and Time Constraints**:
   - `Date` in the **Transaction** entity should follow a valid date format.
   - `Time` in the **Transaction** entity should follow a valid time format.
7. **Quantity Constraints**:
   - `Quantity` in the **Transaction** entity must be a positive integer.

8. **Entity Relationship Constraints**:
    - A student must be associated with at least one transaction.
    - Each transaction must be associated with one student, one mess, and one meal.

Assumptions made while making the ER Diagram-

In the context of the diagram:

- A potential **weak entity** could be **Transaction** if we assume that a transaction is uniquely identifiable only by combining its `TransactionID` with the `MessID` or `StudentID`.
- **Feedback** might be considered a weak entity if it's assumed that feedback can only be uniquely identified by the `TransactionID` and not by its own `FeedbackID`.

Assumptions:

- **Student**, **Mess**, and **Meal** remain strong entities.
- **Transaction** is treated as a weak entity that is dependent on both `Mess` and `Student` for unique identification.
- **Feedback** is considered a strong entity, identified uniquely by `FeedbackID` and related to a specific transaction.

KEYS

Student:

- **Primary Key (PK)**: `StudentID`
- **Explanation**: The `StudentID` uniquely identifies each student in the system. It serves as a primary key to differentiate between individual students.

Mess:

- **Primary Key (PK)**: `MessID`
- **Explanation**: The `MessID` uniquely identifies each mess facility in the system. It is the primary key for the `Mess` table.

Meal:

- **Primary Key (PK)**: `MealID`
- **Explanation**: The `MealID` uniquely identifies each meal served in the mess. It acts as the primary key for the `Meal` table.

Transaction (Weak Entity):

- **Primary Key (PK)**: `TransactionID` (along with `StudentID` and `MessID`)
- **Foreign Keys (FK)**: `StudentID`, `MessID`
- **Explanation**: The `Transaction` table is a weak entity, meaning it cannot exist without its parent entities, `Student` and `Mess`. The composite key consists of `TransactionID`, `StudentID`, and `MessID`, ensuring that each transaction is uniquely associated with a student and a mess. `StudentID` and `MessID` are foreign keys referencing the `Student` and `Mess` tables respectively.

Feedback:

- **Primary Key (PK)**: `FeedbackID`
- **Foreign Key (FK)**: `TransactionID`
- **Explanation**: The `FeedbackID` uniquely identifies each feedback entry. The `TransactionID` serves as a foreign key to link the feedback to a specific transaction, ensuring that feedback is tied to a particular transaction in the mess system.

Relationships:

1. **Assigned_To** (between `Student` and `Mess`):
    - **Foreign Keys (FK)**: `StudentID`, `MessID`
    - **Explanation**: The `Assigned_To` relationship represents the many-to-many relationship between `Student` and `Mess`. This is implemented by using both `StudentID` and `MessID` as foreign keys to reference the `Student` and `Mess` tables, respectively. There is no primary key since this is purely a relationship table.
2. **Eats** (between `Student` and `Meal`):
    - **Foreign Keys (FK)**: `StudentID`, `MealID`
    - **Explanation**: The `Eats` relationship defines that one student can eat multiple meals, but each meal is eaten by one student. `StudentID` and `MealID` are foreign keys that refer to the `Student` and `Meal` tables, respectively.
3. **Serves** (between `Mess` and `Meal`):
    - **Foreign Keys (FK)**: `MessID`, `MealID`
    - **Explanation**: This relationship shows that a mess serves multiple meals, but each meal is served by one mess. `MessID` and `MealID` are foreign keys to the `Mess` and `Meal` tables, respectively.
4. **Has** (between `Meal` and `Transaction`):
    - **Foreign Keys (FK)**: `MealID`, `TransactionID`

- **Explanation**: The `Has` relationship indicates that a meal can have multiple transactions, but each transaction is tied to a single meal. `MealID` and `TransactionID` are foreign keys that reference the `Meal` and `Transaction` tables.

5. **Includes** (between `Transaction` and `Feedback`):
   - **Foreign Keys (FK)**: `TransactionID`, `FeedbackID`
   - **Explanation**: The `Includes` relationship links each transaction to feedback. A transaction can include multiple feedback entries, but each feedback entry belongs to only one transaction. `TransactionID` and `FeedbackID` are foreign keys that reference the `Transaction` and `Feedback` tables.

ER Diagram:

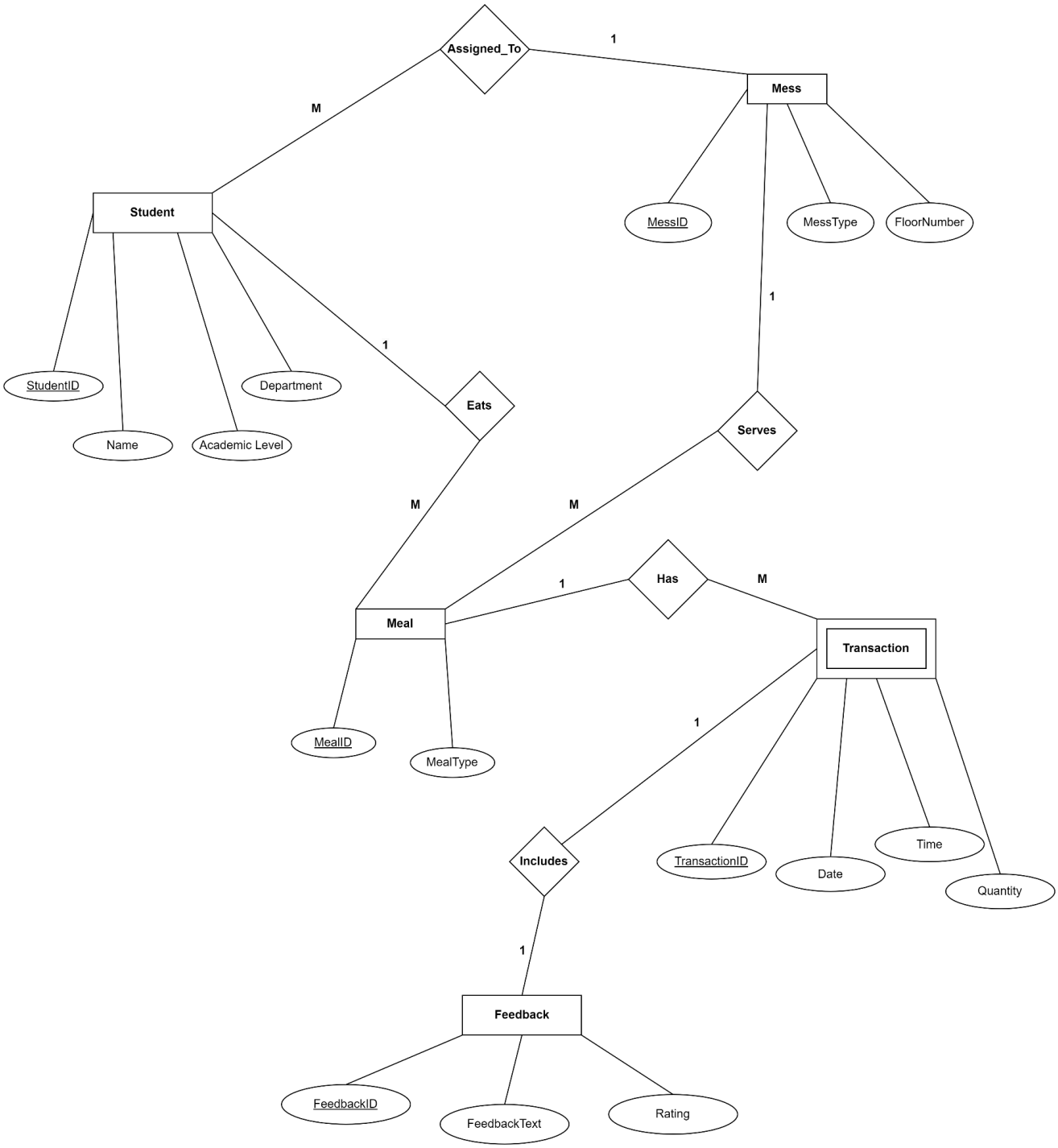


Table Representation:

